AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Postgraduate Programs
Spring 2026

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

Assignment-06

Objective

The goal of this assignment is to explore and implement Counting Semaphores in FreeRTOS, specifically focusing on their application for resource management in a constrained system.

Description

While Binary Semaphores are used for synchronization, Counting Semaphores are ideal for managing a pool of identical resources. You are required to design a FreeRTOS application that utilizes a Counting Semaphore to manage a limited number of shared resources (e.g., 2 or 3 resources) being accessed by a larger number of competing tasks.

You are free to define the exact nature of the resource in your application (e.g., printers, parking spots, shared memory buffers, or global variables). This requires creative engineering thinking to design a valid use case.

Implementation Guide: "The Shared Resource Pool"

To successfully build this scenario, follow these architectural guidelines:

1. **The Limitation:** Imagine a system with **5 worker tasks** but only **2 shared resources** (e.g., 2 UART channels, 2 hardware printers, or a limited memory pool).
2. **Initialization:** Initialize a Counting Semaphore with a maximum count of 2.
3. **Task Execution:** Inside each of the 5 worker tasks' infinite loops, they must:
    - Attempt to take the semaphore (xSemaphoreTake).
    - Once acquired, print a message indicating they claimed the resource.
    - Simulate "using" the resource by blocking for a random amount of time (e.g., using vTaskDelay).
    - Release the resource (xSemaphoreGive) and print a message indicating it was freed.

What your report must demonstrate

- A description of the scenario you engineered: What are the shared resources, how many are available, and how many tasks are competing for them?
- Supporting technical evidence (e.g., UART terminal output, debug logs, or variable watches) clearly showing the semaphore count going down (as resources are claimed) and going up (as resources are freed) in real-time.
- You must prove that **never more than 2 tasks** are inside the "using resource" state at the exact same time, and that the remaining tasks are successfully held in the Blocked state until a token is freed.

Submission format

Submit ONE PDF document containing your scenario explanation and the screenshot evidence proving the dynamic incrementing and decrementing of the semaphore count.